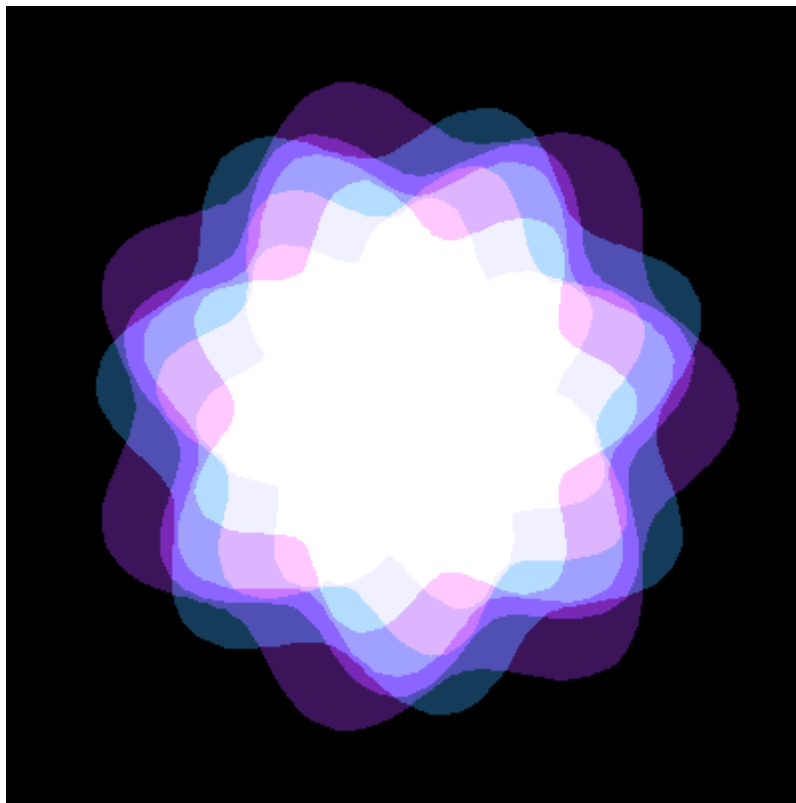


Cours 16 — Trigonométrie : fleurs en coordonnées polaires

Fichiers : ofApp16.h + ofApp16.cpp **Avant :** cours 13 (angle, rayon, cos, sin), cours 04 (std::vector).

Quatorze fleurs emboîtées qui tournent à des vitesses différentes. Une fleur est un cercle dont le rayon **ondule** le long du tour. Pour le dessiner il faut une forme libre, point par point.



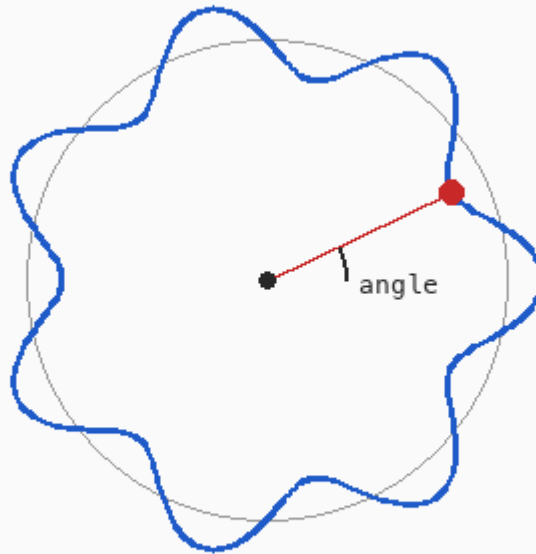
■ 1. Coordonnées polaires

Jusqu'ici un point était (x, y) . On peut aussi le décrire par $(\text{angle}, \text{rayon})$: dans quelle direction, à quelle distance du centre. Ce sont les coordonnées **polaires**. Le passage vers (x, y) est la formule du cours 13 :

```
float x = rayon * cos(angle);  
float y = rayon * sin(angle);
```

L'intérêt : certaines formes sont simples en polaire et compliquées en cartésien. Un cercle, c'est « rayon constant ». Une spirale, « rayon qui grandit avec l'angle ». Une fleur :

■ 2. Un rayon qui ondule



rouge : rayon = $r + \text{amplitude} * \cos(\text{angle} * \text{period})$
gris : cercle de rayon r bleu : la fleur (period = 7 pétales)
 $x = \text{rayon} * \cos(\text{angle})$ $y = \text{rayon} * \sin(\text{angle})$

```
float angle = i * TWO_PI / nbPoints;
float fRadius = amplitude * cos(angle * period);
float x = (r + fRadius) * cos(angle + rotation);
float y = (r + fRadius) * sin(angle + rotation);
```

Le rayon n'est plus r mais $r + \text{amplitude} * \cos(\text{angle} * \text{period})$. Quand l'angle fait un tour, $\text{angle} * \text{period}$ fait period tours, donc le cosinus oscille period fois : period bosses, period pétales. amplitude est leur hauteur. Le $+ \text{rotation}$ fait tourner la fleur entière.

Change period : 3, 5, 12. Change amplitude : 5, 40. Tu contrôles la fleur avec deux nombres.

■ 3. Une forme libre : point par point

```
ofBeginShape();
for (int i = 0; i < nbPoints; i++) {
    // ... calcul de x, y
    ofVertex(x, y);
}
ofEndShape(true);
```

`ofDrawCircle` et compagnie dessinent des formes toutes faites. Pour une forme quelconque, on donne ses sommets un par un : `ofBeginShape()` ouvre la forme, chaque `ofVertex(x, y)` ajoute un sommet,

`ofEndShape(true)` la ferme (le `true` relie le dernier sommet au premier) et la remplit. Avec 200 sommets, la courbe paraît lisse.

■ 4. Plusieurs fleurs, plusieurs vitesses

```
rotations = std::vector<float>(nbFlowers, 0.0f);
```

Une liste de 14 rotations, une par fleur, toutes à 0 au départ. Cette écriture crée directement une liste de `nbFlowers` éléments valant `0.0f`, au lieu de 14 `push_back`.

```
for (int f = 0; f < nbFlowers; f++) {  
    float sens = (f % 2 == 0) ? 1.0f : -1.0f;  
    float speed = f * 0.12f * sens;  
    rotations[f] = rotations[f] + speed * dt;  
}
```

Chaque fleur a sa vitesse : proportionnelle à `f`, donc les fleurs extérieures (petit `f`) tournent lentement et les intérieures vite. Le `%` du cours 13 alterne le sens.

La ligne `sens` utilise un raccourci : `condition ? valeurSiVrai : valeurSiFaux`. C'est un `if` qui tient sur une ligne et **renvoie une valeur**. Équivalent de :

```
float sens;  
if (f % 2 == 0) sens = 1.0f;  
else          sens = -1.0f;
```

Dans `draw()`, la fleur `f` est dessinée avec le rayon `150 - f * 10` et la rotation `rotations[f]` : quatorze cercles concentriques qui deviennent des fleurs.

■ 5. Ce que le mode additif fait ici

Deux couleurs sombres alternées, en `OF_BLENDMODE_ADD` (cours 13) : là où les fleurs se recouvrent, les couleurs s'ajoutent et s'éclaircissent. Les recouvrements deviennent lisibles alors que des formes opaques se cacheraient mutuellement.

■ Exercice final : loupe et tourbillon sur une image

Le cours 14 avait laissé de côté deux effets, parce qu'ils demandent exactement ce que tu viens d'apprendre. Avec la lecture « à l'envers » du cours 14, pour chaque pixel `(x, y)` du résultat :

1. **Cartésien vers polaire** autour de la souris : `dx = x - mouseX`, `dy = y - mouseY`, `r = sqrt(dx*dx + dy*dy)`, `angle = atan2(dy, dx)`. `atan2` fait l'inverse de `cos` / `sin` : il retrouve l'angle d'un vecteur.

2. **Modifier** : loupe, $r = r * 0.5f$ si $r < R$; tourbillon, $angle = angle + k * (1 - r / R)$ si $r < R$.
3. **Polaire vers cartésien** : $sx = mouseX + r * \cos(angle)$, $sy = mouseY + r * \sin(angle)$.
4. **Lire** la source en (sx, sy) avec la fonction bornée, écrire dans le résultat en (x, y) .

Le $1 - r / R$ du tourbillon est la proportion du cours 12 : rotation maximale au centre, nulle au bord du disque, pour raccorder sans cassure.

■ Exercices

1. Une spirale : $rayon = angle * 10$, avec un angle qui va de 0 à $6 * TWO_PI$. Utilise `ofNoFill()` avant `ofBeginShape`.
2. Fais pulser `amplitude` avec le temps.
3. Une seule fleur dont `period` change avec $mouseX / 50$: entier, donc `(int)`.
4. Remplis chaque fleur d'une teinte selon `f` avec `ofColor::fromHsb`.
5. L'exercice final ci-dessus. Commence par la loupe, sans animation.

■ Le code complet, pas à pas

Le programme entier, dans l'ordre des fichiers. Les blocs ci-dessous mis bout à bout donnent exactement `ofApp16.cpp`.

Le fichier `ofApp16.h`

La table des matières du programme : les blocs qui existent et les variables partagées entre eux, avec leurs valeurs de départ.

```
#pragma once
#include "ofMain.h"

// 16 - Trigonométrie : fleurs en coordonnées polaires
// Sketch d'origine : Cours 5/TrigoFlower
// Notions : coordonnées polaires (angle, rayon) -> cartésiennes, modulation du rayon
//          par un cosinus (pétales), forme libre ofBeginShape / ofVertex / ofEndShape,
//          plusieurs instances animées à des vitesses différentes.
//          Exercice final : loupe et tourbillon sur image (polaire -> cartésien appliqué à 14)
// Correction : le sketch d'origine utilisait "flower_index" au lieu de "f_index" dans draw()

class ofApp : public ofBaseApp {
public:
    void setup();
    void update();
    void draw();

    void drawFlower(int fIndex, float r, float speed);

    int    nbPoints = 200;
    float amplitude = 15;    // hauteur des pétales
    int    period = 7;       // nombre de pétales
    int    nbFlowers = 14;
    std::vector<float> rotations;
};
```

En tête du fichier `ofApp16.cpp`

L'inclusion du `.h`, qui rend visibles les variables partagées et les blocs déclarés.

```
#include "ofApp16.h"
```

Étape 1 — `setup()`

Exécuté une fois au lancement : la fenêtre, les chargements, les valeurs de départ.

```
void ofApp::setup() {
    ofSetWindowShape(400, 400);
    // Une rotation par fleur, toutes à 0 au départ.
    // Python : rotations.append(0) dans une boucle. C++ : constructeur avec taille + valeur.
    rotations = std::vector<float>(nbFlowers, 0.0f);
}
```

Étape 2 — `update()`

Exécuté à chaque frame, avant le dessin : tout ce qui change.

```
void ofApp::update() {
    float dt = ofGetLastFrameTime();
    for (int f = 0; f < nbFlowers; f++) {
        // Une fleur sur deux tourne dans l'autre sens
        float sens = (f % 2 == 0) ? 1.0f : -1.0f;
        // Le sketch ajoutait f * 0.002 par frame => f * 0.12 rad/s
        float speed = f * 0.12f * sens;
        rotations[f] = rotations[f] + speed * dt;
    }
}
```

Étape 3 — `drawFlower()`

Une fleur = un polygone de nbPoints sommets.

```
// Une fleur = un polygone de nbPoints sommets.
// Pour chaque sommet : un angle régulier, et un rayon r modulé par un cosinus.
void ofApp::drawFlower(int fIndex, float r, float rotation) {
    ofBeginShape();
    for (int i = 0; i < nbPoints; i++) {
        float angle = i * TWO_PI / nbPoints;
        // cos(angle * period) oscille "period" fois sur un tour => "period" pétales
        float fRadius = amplitude * cos(angle * period);
        // Polaire -> cartésien
        float x = (r + fRadius) * cos(angle + rotation);
        float y = (r + fRadius) * sin(angle + rotation);
        ofVertex(x, y);
    }
    ofEndShape(true); // true = fermer la forme (CLOSE en Processing)
}
```

Étape 4 — `draw()`

Exécuté à chaque frame, après `update()` : uniquement du dessin.

```

void ofApp::draw() {
    ofBackground(0);

    ofPushMatrix();
    ofTranslate(ofGetWidth() / 2, ofGetHeight() / 2);

    // Processing : blendMode(DIFFERENCE), qui inverse là où les fleurs se recouvrent.
    // Sans équivalent direct en openFrameworks : on alterne deux couleurs en ADD,
    // ce qui rend les recouvrements lisibles. (Un vrai DIFFERENCE se fera plus tard en shader.)
    ofEnableBlendMode(OF_BLENDMODE_ADD);
    ofFill();
    for (int f = 0; f < nbFlowers; f++) {
        if (f % 2 == 0) ofSetColor(60, 20, 90);
        else             ofSetColor(20, 60, 90);
        drawFlower(f, 150 - f * 10, rotations[f]);
    }
    ofEnableBlendMode(OF_BLENDMODE_ALPHA);

    ofPopMatrix();

    // Alternative moderne à ofBeginShape : construire une ofPolyline
    // (polyline.addVertex(x, y) ... polyline.close() ; polyline.draw())
    // ou un ofPath pour un remplissage. À montrer quand on aborde les objets.

    // Exercice (image) : loupe et tourbillon, avec la lecture "à l'envers" de 14.
    // Pour chaque pixel (x, y) du résultat :
    // 1. cartésien -> polaire autour de la souris : dx = x - mouseX, dy = y - mouseY,
    //    r = sqrt(dx * dx + dy * dy), angle = atan2(dy, dx)
    // 2. modifier : loupe = r * 0.5 si r < R ; tourbillon = angle + k * (1 - r / R) si r < R
    // 3. polaire -> cartésien : sx = mouseX + r * cos(angle), sy = mouseY + r * sin(angle)
    // 4. lire la source en (sx, sy), écrire dans le résultat en (x, y)
}

```

■ Ce qu'il faut retenir

- Polaire (angle, rayon) vers cartésien : $x = \text{rayon} * \cos(\text{angle})$, $y = \text{rayon} * \sin(\text{angle})$.
Inverse : $\text{rayon} = \sqrt{dx*dx + dy*dy}$, $\text{angle} = \text{atan2}(dy, dx)$.
- Une fleur : $\text{rayon} = r + \text{amplitude} * \cos(\text{angle} * \text{period})$, period pétales.
- `ofBeginShape(); ofVertex(x, y); ... ofEndShape(true);` dessine une forme libre.
- `std::vector<float>(n, valeur)` crée une liste de n éléments.
- `condition ? a : b` : un `if` qui renvoie une valeur.